

CAPITULO ESTUDIANTEL ACM - ICPC UMSA

1

CAMPAMENTO DE PROGRAMACIÓN COMPETITIVA

ENERO 2019





UNIV. CARLOS FERNANDO TORREZ ALANOCA
UNIV. RODRIGO JAVIER CASTILLO GUZMAN
UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES





1. Cadenas



- 1 Z Algorithm
 - Función Z
 - Algoritmo Trivial
 - Algoritmo Eficiente
 - Aplicaciones

- 2 KMP
 - Bordes
 - Función de Prefijos
 - Algoritmo KMP

- 3 Trie



1

Z Algorithm



UMSA
la mejor

Definiciones



Dada una cadena: $S = S_0 S_1 S_2 \dots S_{n-1}$

Donde: $|S| = n$

El prefijo de la cadena S hasta el índice i se denota:

$$S_0 S_1 \dots S_i$$

El sufijo de la cadena S desde el índice i se denota:

$$S_i S_{i+1} \dots S_{n-1}$$

Nota: los prefijos y sufijos propios no contemplan a toda la cadena.



Función Z



la función Z de una cadena esta definida para cada índice de la siguiente manera.

$$Z_i = \begin{cases} 0, & \text{si } i = 0 \\ \text{tamaño del prefijo común mas grande entre } S \text{ y } S[i..n-1] \end{cases}$$

Ejemplo:

$S =$	A	B	A	C	A	B	A
$Z =$	0	0	1	0	3	0	1



Algoritmo Trivial



la forma mas simple de obtener la función Z de una cadena S en cada índice i es ir buscando el prefijo común mas largo entre S y $S[i \dots n - 1]$. Como se puede ver la complejidad seria $O(|S|^2)$



Algoritmo Trivial



Se inicia colocando el valor inicial 0 a cada valor i de Z

```
1  for(int i = 0; i < n; i++){
2      while(Z[i] + i < n && S[Z[i] + 1] == S[Z[i]]){
3          Z[i]++;
4      }
5  }
```



Algoritmo Eficiente



- Se usa la misma idea del **Algoritmo Trivial** solo que esta vez se reusan algunos datos.
- La idea es tener un rango que inicia en l y termina en r que sea igual a un prefijo de S donde $Z[l] = r - l + 1$.
- Ahora supongamos que queremos calcular el valor de Z en el índice i tenemos 2 opciones que i este dentro de nuestro rango entre l y r o este fuera.
- Si esta dentro el rango i tiene su reflejo i' en el prefijo de S que ya esta calculado su valor Z entonces el valor $Z[i]$ es igual al mínimo entre $Z[i']$ y $r - i + 1$ que es el limite de nuestro rango.
- buscamos agrandar lo mas que podamos el valor de $Z[i]$, si es asi actualizamos los valores de l y r .



Algoritmo Eficiente



```
1  l = r = 0;
2  for(int i = 0; i < n; i++){
3      if(i <= r){
4          Z[i] = min(Z[i - 1], r - i + 1);
5      }
6      while(Z[i] + i < n && S[Z[i] + 1] == S[Z[i]]){
7          Z[i]++;
8      }
9      if(r < i + Z[i] - i){
10         l = i;
11         r = i + Z[i] - i;
12     }
13 }
```





- String Matching Problem
- contar la cantidad diferentes de subcadenas de cualquier tamaño
- Muchas mas ...



2

KMP



UMSA
la mejor



Dada una cadena: $S = S_0 S_1 S_2 \dots S_{n-1}$

Se define borde como un prefijo propio igual a un sufijo propio de la cadena S .

Por ejemplo a y $abra$ son bordes de $abracadabra$.



Función de Prefijos



La función de prefijos esta definida de la siguiente manera:

$PF[i] = \text{tamaño del borde mas grande de } S[0...i - 1]$

- $PF[i] \leq PF[i - 1] + 1$
- $PF[PF[i]]$ es el tamaño del segundo mayor borde de la cadena $S[0....i - 1]$



Función de Prefijos



```
1
2  for(int i = 2; i <= n; i++){
3      PF[i] = PF[i - 1];
4      while(PF[i] > 0 && S[i - 1] != S[P[i]]){
5          PF[i] = PF[ PF[i]];
6      }
7      if(S[P[i]]==S[i-1]){
8          PF[i]++;
9      }
10 }
```



Algoritmo KMP



- Para el problema de String Matching de P en T se puede calcular la Función de prefijos sobre la cadena $S = P\#T$.
- Se puede notar que $PF[i] \leq |P|$ para cualquier posición de S
- El algoritmo de Knuth-Morris-Pratt (también conocido como KMP) consiste en 2 fases.
- Como se dijo antes solo se necesita preprocesar la función de prefijos de la cadena P la primera fase consiste en eso.
- La segunda fase consiste en ejecutar un algoritmo parecido al de la Función de prefijos sobre la cadena T sin guardar datos y utilizando los datos de la Función de Prefijos de la cadena P .



Algoritmo KMP



```
1 k = 0;  
2 //m es el tamaño de T  
3 for(int i = 0; i < m; i++){  
4     while(k > 0 && P[k] != T[i]){  
5         k = PF[k];  
6     }  
7     if(P[k]==T[i]){  
8         k++;  
9     }  
10 }
```



3

Trie



UMSA
la mejor



Un Trie es una estructura de datos de tipo árbol rooteado, en la cual se pueden almacenar cadenas descomponiéndolas carácter por carácter en forma descendente creando nodos si nos hace falta.





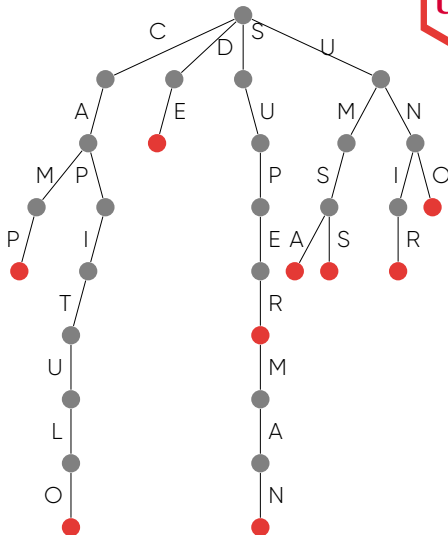
Un Trie es una estructura de datos de tipo árbol rooteado, en la cual se pueden almacenar cadenas descomponiéndolas carácter por carácter en forma descendente creando nodos si nos hace falta.



Trie



- CAMP
- CAPITULO
- DE
- SUPER
- SUPERMAN
- UMSA
- UMSS
- UNIR
- UNO



The background consists of several overlapping, three-dimensional rectangular blocks in various shades of red and pink. The blocks are arranged in a way that creates a sense of depth and perspective, with some blocks appearing to be in front of others. The colors range from a deep, dark red to a very light, almost white pink.

¿Preguntas?

The background consists of several overlapping, semi-transparent 3D rectangular blocks in various shades of red and pink. The blocks are arranged in a way that creates a sense of depth and perspective, with some blocks appearing to be stacked on top of others. The lighting is soft, creating subtle gradients and shadows on the surfaces of the blocks.

Gracias!